

Experiment No. 2

Topic: Agile Requirements Gathering with User Stories (For **NON-Functional Requirement**) **Aim:** To identify Nonfunctional requirement for any software development project.

Learning Outcome: Study consists of learning the following concepts

- Identify non-functional requirements for the software to be designed in each context.
- Write effective User stories for the non-functional / Quality requirements of selected projects.
- Acceptance criteria and Definition of done.
- Incorporating Design decisions to achieve desired quality for software systems.

Part A:

Functional vs Non-Functional Requirements

S. No	Functional	Non-Functional
1	Defines 'What' of the customer requirements	Defines 'How' of the customer requirements
2	Elicited by business analysts, specified by the customer	Defined by technical team like architects, developers etc
3	Documented as the requirements document by the business analyst	Documented as the technical document by the team
4	Defines the functionality in the system	Defines quality attribute in the system
5	Captured as a user story in agile	Captured either as a story or part of acceptance criteria of a user story
6	Customer tests and validates the functionality as specified by them	Customers cannot test these quality attributes, however, can experience while using the system
7	Customers find it easy to define these	It is hard to define by the customer

S. No	Functional	Non-Functional
8	Defines the product features	Defines the product properties

Non-Functional / Quality requirements for Software

- **Usability.** The higher the usability of software, the easier it is for users to work with it. There are several aspects of usability, including learnability for novices, efficiency of use for experts, and handling of errors.
- **Efficiency.** The more efficient software is, the less it uses of CPU-time, memory, disk space, network bandwidth and other resources. This is important to customers in order to reduce their costs of running the software, although with today's powerful computers, CPU-time, memory and disk usage are less of a concern than in years gone by.
- **Reliability.** Software is more reliable if it has fewer failures. Since software engineers do not deliberately plan for their software to fail, reliability depends on the number and type of mistakes they make. Designers can improve reliability by ensuring the software is easy to implement and change, by testing it thoroughly, and by ensuring that if failures occur, the system can handle them or can recover easily.
- **Maintainability.** This is the ease with which you can change the software. The more difficult it is to make a change, the lower the maintainability. Software engineers can design highly maintainable software by anticipating future changes and adding flexibility. Software that is more maintainable can result in reduced costs for both developers and customers.
- **Reusability.** A software component is reusable if it can be used in several different systems with little or no modification. High reusability can reduce the long-term costs faced by the development team

How to Elicit Non-Functional Requirements?

When the business analysts elicit requirements from the stakeholders on the functional aspects of the system, they also should understand from the stakeholder what attributes the system should provide in order to meet their business goals. For example, the website must be able to load within 0.5 seconds. Eliciting a non-functional requirement is more challenging than the functional requirement, as the stakeholders/users are good at articulating the functional requirements well; while they may not be aware of, or good at explicitly stating non-functional requirements. Business analysts will have to get these requirements from stakeholders by asking the right questions. NFRs are implicit by nature, meaning that people assume them to exist without being asked.

Some of the questions to be prompted that may help the business analyst to ask stakeholders how they want their system to function include the following aspects:

- **Performance** – speed of the system e.g., response time, throughput etc
- **Availability** – what is the impact if the system is not available for the customer?
- **Security** – ensuring unauthorized access to the system
- **Data Retention** - how long should the data be retained in the system for reference? This could be a regulation from the government/country.
- **Usability** – easy access for the customer/user while using the system
- **Stability** – code/system stability when dynamic changes apply due to business needs
- **Compliance** – understanding compliance needs by local laws/regulations
- **Reliability** – probability of system performing without failure
- **Recoverability** – ability of the system to recover from failure
- **Serviceability** – ease of system service when necessary
- **Data integrity** – accuracy and correctness of data maintained in the system
- **Scalability** – ability of the system to scale when more resources are added
- **Capacity** – how much the system can store information
- **Accessibility** - how easily people with the widest range of capabilities can use the system.

Non-Functional Requirements in Agile

A common challenge for an Agile team is dealing with capturing non-functional requirements in a user story. The non-functional requirements can be written as a user story and made available in the product backlog or sprint backlog. For example, “As an ecommerce website user, I want the website to be available for 99.99999%, so that I purchase products whenever I feel like”

NFR can also be included as Acceptance Criteria in a user story.

For example, for the following user story, “As an ecommerce website user, I want to search for products, so that I can purchase them”. NFR as acceptance criteria would be “application responds to search request within 5 seconds from the time request is received”.

Examples of non-functional requirements:

- As a customer, I want to be able to run your product on all versions of Windows from Windows 95 on.
- As the CTO, I want the system to use our existing orders database rather than create a new one, so that we don't have one more database to maintain.
- As a user, I want the site to be available 99.999 percent of the time I try to access it, so that I don't get frustrated and find another site to use.
- As someone who speaks a Latin-based language, I might want to run your software someday.
- As a user, I want the driving directions to be the best 90 percent of the time, and reasonable 99 percent of the time.

Few More examples:

- **Performance** – funds transfer transaction should not take more than 0.5 seconds

- **Availability** – the system should be available for the users to use the system anytime of the week they need
- **Security** – payroll data is not visible to all the employees in the organization. Only the authorized personnel should have access to the data
- **Data Retention** – all healthcare data should reside in the system for 7 years. Older data can be archived and should be accessible when required
- **Usability** – the system navigation is easy for the end user and provides a seamless journey while accessing the system
- **Stability** – the code/system is stable when new changes are applied due to dynamic business needs
- **Compliance** – HIPAA compliance should be adhered when dealing with patients' data in USA
- **Reliability** – website users should be able to access the system 99% of the time without any failure
- **Recoverability** – In case of any major failures, the system should be restored within 48 hours
- **Serviceability** – the system should be serviceable immediately when there is a failure
- **Data integrity** – the system should not allow phone number to be entered in the data field
- **Capacity** – the system should be able to store 1 million records of basic customer information
- **Scalability** – the system should be scalable to support unlimited growth of the employees
- **Accessibility** - The system shall be accessible to people with disabilities in accordance with the Americans with Disabilities Act
- **Confidentiality** – The system shall not display the bank account number in full. It should display only the last 4 digits of the account number while others can be masked
- **Efficiency** – The interface between the system and user should not take more than 2 seconds
- **Portability** – The system shall be developed for both windows and Mac operating systems platforms

For performance requirements and many other nonfunctional requirements(NFR's), one can use constraints and stories.

- Usually create a story to document the NFR and
- Define story tests for it.
- Then, add the story tests as a "constraint."

A constraint is something that all implemented stories(features and functionality) must comply with.

SAMPLE:

Step 1: Identify and quantify the constraint, put it in terms that your users and business stakeholders will understand, and put it into one or more stories as Story Tests (aka Acceptance Tests).

Story Title: System response time

- **Story Test #1:** Test that the system responds to all non search requests within 1 second of receiving the request
- **Story Test #2:** Test that the system responds to all search requests within 10 seconds of receiving the request

Some things to keep in mind:

- If you cannot quantify the story in concrete terms, this should be a bad smell that usually indicates a requirement that is too vague to be implemented. Vague NRF's have the same problems that vague functional requirements do: It is hard to answer the question "How will I know when this story is correctly done?"
- Be sure not to specify a technical solution or implementation in the story, because stories are about **"The What"** ("What the user wants") and they are not about **"The How"** ("How this is implemented").
- Plan, estimate, split (if necessary), and implement this story like all other user stories, as part of the Product Backlog (Scrum).

Once this story is complete, the entire system will be in compliance with this particular constraint.

If your constraint is not system-wide or far reaching:

- Just add it as a story test for that story. But again, specify the requirement, not the implementation, in terms the business stakeholders will understand.

The decision to create a constraint or not will rest on whether the constraint should be considered in at least several future stories (or system wide).

If it will apply to several future stories, then create a constraint.

If it won't apply to several future stories, then just add the NFR as a story test to the stories that it applies to or create a separate story to comply with the NFR in the small part of the system that requires it.

Step 2: Add the NFR related Story Tests to your list of constraints (and to your Definition of Done if you're doing Scrum)

Publish your list of constraints (and/or DoD) somewhere that is highly visible. Even if you keep your constraints electronically, print them out in large print and post them somewhere on your Scrum board or in your team area.

Constraints
• Test that the system responds to all non search requests within 1 second of receiving the request.
• Test that the system responds to all search requests within 10 seconds of receiving the request.
• Test that the system logs a user out after 10 seconds of inactivity and redirects their browser to the home page.
• Test that any update to a person's payment information(anywhere in the system) is logged to the payment_preferences log, along with the following information: ◦ IP Address of logged in person ◦ Old preference value, new preference value ◦ Date/time of change
• Test that any time a person's credit card number is shown in the application, that only the last 4 digits display.

Add the constraints to your Definition of Done.

Definition of Done
• All stories must comply with all of the story constraints<link to constraints page on wiki>.
• All code must be peer reviewed within 4 hours of checkin.
• If a change is made to the web services interface, the change must be documented on the official web services api wiki page<link to api on wiki>.
• All code must have automated testing that is consistent with the "Automated Testing Guidelines"<link to guidelines on wiki>
• Any change in of functionality that is visible in the GUI must be at least tested manually(automated tests also acceptable) against the integration environment before making the functionality available for a QA review.

Part B:

Functional Requirement	Non-Functional Requirement
Register new members	Performance: Member registration should be completed within 3 seconds.
Verify member details	Data Integrity: The system should validate all member information before saving.
Schedule trial sessions	Availability: The scheduling system should be available 99.9% of the time.
Print payment receipts	Reliability: Payment receipts should be generated accurately without errors.
Check member attendance	Usability: Attendance records should be easy to access and understand.
Assign workout plans	Performance: Workout plans should load within 2 seconds.
Record member progress	Reliability: Member progress should be saved accurately without data loss.

Schedule personal training sessions	Availability: Training schedules should always be accessible during working hours.
Update exercise routines	Maintainability: Exercise routines should be easy to update without affecting existing data.
View member medical restrictions	Security: Medical information should only be accessible to authorized trainers.
Approve new staff accounts	Security: Only administrators should be able to approve staff accounts.
Manage gym branches	Scalability: The system should support multiple gym branches efficiently.
Maintain equipment schedules	Reliability: Equipment maintenance records should never be lost.
Monitor daily gym attendance	Performance: Attendance reports should be generated within 5 seconds.
Monitor department activities	Availability: The monitoring dashboard should be available 99.9% of the time.
Create membership plans	Usability: Membership plans should be easy to create and modify.
Cancel inactive memberships	Data Integrity: Only valid inactive memberships should be cancelled.
Renew memberships	Performance: Membership renewal should be completed within 2 seconds.
Send renewal reminders	Reliability: Renewal reminders should be delivered successfully.
Track membership status	Availability: Membership status should be available whenever requested.
Collect membership payments	Security: Payment information should be encrypted and accessible only to authorized staff.
Generate invoices	Reliability: Invoices should be generated accurately every time.
Review revenue reports	Performance: Revenue reports should be generated within 5 seconds.
Process refunds	Data Integrity: Refund transactions should maintain accurate financial records.
Track overdue payments	Availability: Overdue payment information should be available whenever required.

- Create Document of NFR for your selected Software application.

— GYM OPERATIONS PLATFORM

ONE SYSTEM. EVERY DEPARTMENT ON THE SAME FLOOR.

Ironboard connects reception, training, administration, membership and accounting into a single board — so no member, session or payment falls through the cracks.

[EXPLORE DEPARTMENTS](#)[SEE HOW IT WORKS](#)**05**

CORE DEPARTMENTS

25

TRACKED USER STORIES

01

SHARED SOURCE OF TRUTH

DEPARTMENTS

BUILT AROUND WHO
ACTUALLY RUNS THE GYM.

Each department gets its own board, scoped to the work its role owns — from the front desk to the ledger.

DEPT / 01 5 stories RECEPTION The first touchpoint — registration, verification and the daily front-desk rhythm. — Register new members & issue Member IDs — Verify and update member details — Schedule trial sessions for prospects — Print payment receipts — Check daily member attendance	DEPT / 02 5 stories TRAINER DEPT. Where fitness goals turn into plans, sessions and measurable progress. — Assign and link workout plans — Record member progress & measurements — Schedule personal training sessions — Update exercise routines — View member medical restrictions	DEPT / 03 5 stories ADMINISTRATION Oversight across staff, branches, equipment and daily operations. — Approve new staff accounts — Manage multiple gym branches — Maintain equipment maintenance schedules — Monitor daily gym attendance — Monitor all department activity	DEPT / 04 5 stories MEMBERSHIP MGMT. The lifecycle of a membership, from sign-up to renewal or lapse. — Create membership plans & packages — Cancel inactive memberships — Renew memberships for continuity — Send renewal reminders — Track active vs. expired status
DEPT / 05 5 stories ACCOUNTING Every payment, invoice and overdue balance, reconciled in one place. — Collect membership payments — Generate invoices for members — Produce revenue reports — Process refunds — Track overdue payments			

GET STARTED

PUT EVERY DEPARTMENT
ON ONE BOARD.

REQUEST A DEMO

TALK TO SALES



DEPARTMENTS

Reception

Trainer

Administration

DEPARTMENTS

Membership

Accounting

PRODUCT

Workflow

Features

© 2026 Ironboard Gym Management System

Built on the reception, trainer, admin, membership & accounting modules

HOW IT FLOWS

FROM WALK-IN TO RENEWAL.

A single member journey, handed cleanly from one department's board to the next.

RECEPTION	MEMBERSHIP	TRAINER	ACCOUNTING	ADMIN
Prospect walks in, trials a session, and reception registers them with a Member ID.	A plan is chosen and activated, with renewal reminders queued automatically.	A workout plan is assigned and progress is logged against the member's profile.	Payments are collected, invoiced, and reconciled against membership status.	Attendance, staff and equipment are monitored across every branch, always.

PLATFORM FEATURES

EVERYTHING EACH ROLE ACTUALLY NEEDS.

No department has to dig through another's tools to get their job done.



Member profiles

One record per member — contact details, plan, attendance and medical notes, visible to the right roles only.



Session scheduling

Trials and personal training sessions booked against real-time slot availability.



Progress tracking

Weight, measurements and workout history charted automatically as trainers log data.



Billing & invoicing

Receipts, invoices, refunds and overdue balances handled without leaving the platform.



Branch oversight

Attendance, equipment schedules and staff accounts monitored across every location.



Renewal alerts

Expiring memberships flagged early, with reminders sent before members lapse.

GET STARTED

PUT EVERY DEPARTMENT ON ONE BOARD.

[REQUEST A DEMO](#)[TALK TO SALES](#)

DEPARTMENTS

Reception

Trainer

Administration

DEPARTMENTS

Membership

Accounting

PRODUCT

Workflow

Features

© 2026 Ironboard Gym Management System

Built on the reception, trainer, admin, membership & accounting modules